

基于 ISO/IEC 文档的多智能体在线语音狼人杀系统软件过程定义

2023141461221 软件学院 沈矜娴

1 ISO/IEC 文档内容理解

1.1 ISO/IEC 12207

ISO/IEC 12207 是关于软件生命周期过程的标准描述，它主要关注一个软件开发过程中应当经历哪些过程，以及需要完成哪些活动和任务。文档提供了软件从需求定义、架构设计、详细设计、实现、集成、测试、交付、运行维护到退役等过程的通用框架，不同组织和项目可以根据自身特点，对其中的过程进行选择、组合和剪裁。

在该标准中，软件生命周期过程通常可以理解为几个层面的集合：一类是与项目获取、供应和交付相关的过程；一类是支持项目顺利开展的管理和组织过程；还有一类是与软件产品本身直接相关的技术过程，覆盖了软件从需求到设计、从实现到测试、从交付到维护的完整链条。通过这种划分，软件开发活动被组织成了一套有职责划分、顺序关系的工程化过程。

1.2 ISO/IEC 15504

ISO/IEC 15504 是软件过程评估领域的重要标准，强调从过程能力角度评价一个软件过程的执行。软件过程不仅要被执行，还应当能够被管理、被评价和被持续改进。文档通常从两个维度来看待软件过程：一是过程维度，即需要评价哪些软件过程，例如需求分析、设计、实现、测试、配置管理、项目管理等；二是能力维度，即某个过程达到了怎样的能力水平。能力水平一般从低到高逐步递进，体现过程从没有完成或执行不足，到能够实现基本目标，再到形成标准并持续优化的发展过程。

因此文档的主要内容是建立了一种评价软件过程能力的方法，使开发者能够了解自身软件过程的现状，发现过程中的不足，并为后续过程改进提供依据。

1.3 ISO/IEC 33001

ISO/IEC 33001 属于进一步发展出的过程评估标准体系，主要提供过程评估的基本概念、术语和总体框架，例如过程、过程属性、过程能力、过程评估和过程改进等。

与 ISO/IEC 15504 相比，该文档更偏向概念性和基础性。它并不重点描述某个具体项目应该如何开发软件，也不直接给出某一类项目的生命周期模型，而是为过程评估活动提供统一的术语体系和理解框架。通过这些概念，软件过程可以被更加规范地描述为具有目标、输入、输出、活动、结果和能力特征的对象，而不同人员在讨论过程能力、过程评价和过程改进时也能使用上一致的语言。

2 项目概述

我所选择的项目是“多智能体在线语音狼人杀系统”，它旨在开发一个支持真人玩家与 AI 玩家同局博弈的在线狼人杀平台。系统通过多智能体 AI 协同、实时语音交互和智能复盘技术，解决传统在线狼人杀中凑局难、AI 拟真性不足和复盘功能薄弱等问题。

项目围绕狼人杀游戏的完整对局流程展开，支持用户通过网页端完成一局完

整的狼人杀游戏。在此基础上，系统为 AI Agent 设计了记忆、发言生成、投票决策和夜晚行动等能力，使其能够在一定程度上模拟真人玩家的思考过程，并在玩家人数不足时补足对局人数，提升游戏的连续性和可玩性。

3 项目软件过程定义

3.1 软件过程总体定义

本项目的软件过程以 ISO/IEC 12207 的生命周期过程思想为基础，结合项目的规模、团队协作方式和系统技术特点，对相关过程进行了适当取舍和调整。由于“多智能体在线语音狼人杀系统”同时涉及实时游戏流程控制、WebSocket 通信、AI Agent 推理决策、语音识别与合成、日志记录和复盘分析等内容，项目中存在较多需要通过实践验证的技术不确定性。因此，将本项目的软件过程定义为一种以演化型原型为主线、逐步完善核心功能为目标的迭代式软件过程。

在该过程中，项目开发并不是一次性完成需求、设计、编码和测试后再交付，而是围绕系统核心功能进行逐步演进。项目团队首先通过项目计划明确开发目标、系统范围、团队分工、进度安排和主要风险；随后通过需求分析梳理用户角色、业务流程、功能需求、接口需求和非功能需求。在此基础上，团队先围绕房间管理、开局流程、昼夜状态切换、投票结算等核心游戏流程构建了初步原型，用于验证系统的基本业务是否成立。随后再进一步完善系统架构、模块边界、数据库结构、接口协议和实时通信机制，并逐步引入 AI Agent 协同游玩、语音交互和 AI 生成复盘分析等复杂功能。

本项目的软件过程具有明显的演化特征：早期原型重点验证游戏流程能否跑通，例如玩家能否创建房间、加入房间、完成角色分配并按照昼夜阶段推进游戏；中期原型则开始关注模块之间的协作，例如前后端逻辑的连接、数据库状态和 WebSocket 消息推送之间的协同；后期原型进一步集成 AI 与语音相关功能，验证 AI Agent 能否基于游戏上下文进行发言、投票和夜晚行动，语音模块能否完成玩家语音输入、文本转写和 AI 语音输出。每一次原型演进都在已有系统基础上补充额外功能，修正问题，使系统不断地更新迭代至全部开发完毕。

与传统瀑布式过程相比，本项目更加重视持续的验证和阶段性改进，同时还注重开发文档的撰写和阶段成果的交付。诸如项目计划书、需求分析文档和系统设计文档都在软件过程中都经过了反复的修改完善，用于约束我们代码的实现和测试活动，避免原型迭代失去方向。因此，本项目的核心是在开发文档的基础上，让系统功能通过多轮从原型构建，功能实现，测试验证到问题修正的循环逐步成熟。

3.2 软件过程阶段划分

阶段	过程名称	过程目标	主要活动	主要输出
1	项目启动与计划	明确项目目标、范围、分工、进度和主要风险	分析项目背景，确定系统目标和功能边界，划分软件开发角色分工；制定进度计划，识别项目风险	项目计划书
2	需求获取与分析	建立需求基线，明确软件要做什么	用户角色分析、功能需求分析、接口需求分析、非功能需求分析、其他需求分析、需求优先级划分	需求分析文档、需求跟踪矩阵

3	原型设计与可行性验证	验证核心业务和关键技术路线，降低后续实现风险	构建房间流程原型、游戏状态机原型，验证 LLM 调用、语音 STT/TTS 接口和 AI 行为生成的可行性	核心原型、技术验证结果
4	系统架构与详细设计	将需求转化为可实现的技术方案	总体架构设计、功能模块设计、接口设计、数据结构设计、性能设计、部署设计	系统设计说明书
5	编码实现与单元验证	逐步实现系统功能并验证单个模块是否可用	前后端连接、游戏逻辑实现、AI Agent 模块实现、语音模块实现、数据库实现、单元测试编写	源代码、单元测试报告
6	集成测试与迭代修正	验证系统是否能够支撑完整游戏流程，并根据问题进行修正	模块集成、接口联调、功能测试、异常测试、性能测试、需求跟踪检查	集成测试记录、缺陷记录、修正结果
7	交付与确认	完成项目验收和系统交付	系统演示、文档提交、代码提交、功能验收	可运行系统、交付文档、验收材料
8	项目总结与过程改进	总结项目执行过程中的问题并提出改进措施	分析需求变更、团队协作、开发进度、测试缺陷和系统不足。提出后续的功能优化方向	项目总结、改进建议

4 软件过程定义来源

4.1 ISO/IEC 12207 的生命周期过程思想

本项目软件过程的基本框架主要来源于 ISO/IEC 12207 的软件生命周期过程思想。ISO/IEC 12207 提供了一套用于组织软件生命周期活动的通用框架，强调软件从需求形成到设计、实现、集成、验证、确认、交付和维护之间存在连续关系，各阶段活动并不是孤立发生的，而是共同服务于软件产品的形成与演进。

在定义本项目的软件过程时，我主要借鉴了该文档中“生命周期过程可以被选择和剪裁”的思想。由于本项目属于课程设计内容，不具备完整商业软件项目中的合同、采购、供应和长期运维条件，因此没有照搬标准中的所有过程，而是选取与项目实际开发最密切相关的活动，形成适合本项目的过程主线。这些活动包括项目计划、需求分析、原型验证、系统设计、编码实现、集成测试、交付确认和总结改进。

其中，项目计划用于明确项目目标、范围、进度、分工和风险；需求分析用于确定系统要实现的功能和质量要求；原型验证用于提前检查核心流程和关键技术是否可行；系统设计用于将需求和原型结果转化为架构、功能模块、接口和数据结构方案；编码实现和集成测试用于逐步形成可运行系统；交付确认和总结改进则用于完成课程验收并积累过程经验。这样的过程安排保留了 ISO/IEC 12207 中生命周期管理的基本逻辑，同时也避免了超出课程项目规模的复杂流程。

4.2 ISO/IEC 15504 的过程能力评估思想

本项目的软件过程定义还参考了 ISO/IEC 15504 的过程能力评估思想。ISO/IEC 15504 关注在项目中某一个过程是否真正发挥了作用，是否能够被管理、

被检查和被改进。主要表达软件过程不能只停留在“做过某项活动”的层面，还应当关注该活动是否有明确目标，是否产生了可检查的结果，以及是否能够为后续阶段提供依据。

基于这一思想，在定义项目过程时，我不只是简单列出“计划、需求、设计、编码、测试”等阶段，而是为每个阶段进一步明确了过程目标、主要活动和主要输出，让每个过程都留下相应的输出和检查依据，例如项目计划书、需求分析文档、软件设计说明书、原型验证结果、测试记录、缺陷记录和改进建议等。通过这些过程资产，我们才可以判断项目过程是否被有效执行，也可以在发现问题后追溯到需求、设计、实现或测试中的具体环节，从而支持后续修正和改进。

4.3 ISO/IEC 33001 的过程概念与术语

ISO/IEC 33001 为过程评估提供了基本概念和术语基础，其中包括过程、过程属性、过程能力、过程评估和过程改进等内容。对于本项目而言，这一规范性的体系有助于避免把软件过程简单理解为按顺序完成若干任务，而是将每个过程看作一个有目标、有输入、有活动、有输出，并且可以被评价和改进的活动集合。

在本文的软件过程定义中，每一阶段的任务都具有明确的边界。每个过程的描述都需要涵盖：该过程要解决什么问题，依赖哪些前置结果，需要完成哪些活动，最终输出什么成果，以及如何判断该过程是否达到预期。这种描述方式使项目的软件过程更加清晰。比如，原型设计过程的目标不是单纯做一个界面样例，而是验证核心游戏闭环和关键技术路线；集成测试过程的目标也不是简单运行程序，而是确认多个模块组合后能否支撑完整对局流程。通过参考文档中关于过程和过程评估的概念，我能够更规范地说明项目软件过程的组成、边界和评价方式。

5 软件过程剪裁说明

在定义“多智能体在线语音狼人杀系统”的软件过程时，本文结合课程设计项目的规模、开发周期、团队组织形式和系统技术特点进行了剪裁。剪裁的基本思路是：保留与项目开发、课程验收和质量保证直接相关的过程，简化需要大型组织支撑的管理过程，同时删除或弱化与项目无关的商业运维化过程。通过这种方式，使软件过程既能够体现生命周期思想，又符合本项目作为课程设计项目的实际情况。

5.1 保留内容

本项目保留了项目计划、需求分析、系统设计、编码实现、集成测试、交付确认和总结改进等核心过程。这些过程与课程项目的目标高度相关，也是软件系统从概念形成到最终交付所必须经历的基本活动。

项目计划过程用于明确系统建设目标、开发范围、人员分工、进度安排和风险控制措施；需求分析过程用于识别玩家、房主、管理员和 AI Agent 等不同角色的需求，并将这些需求转化为可设计、可实现、可测试的系统需求；系统设计过程用于将需求分析结果进一步转化为系统架构、模块划分、接口协议、数据库结构和游戏状态机设计；编码实现过程用于按照设计结果逐步完成前端、后端、AI Agent、语音服务和数据存储等模块；集成测试过程用于验证各模块之间能否正确协同，尤其是游戏流程、实时通信、AI 决策和语音交互等关键功能；交付确认过程用于完成系统演示、文档提交和课程验收；总结改进过程则用于总结项目开发过程中的问题、风险和改进方向。上述过程构成了本项目软件过程的主体框架。

在此基础上，我们对需求分析、原型设计和系统设计过程进行了适当强化。由于狼人杀游戏本身包含复杂的昼夜阶段流转、角色技能约束、投票放逐、胜负判定和玩家状态变化，如果需求分析不充分，后续实现很容易出现规则遗漏或逻辑冲突。同时，我们为了创新还引入了 AI Agent、语音识别与合成、AI 复盘等功能，这些功能之间存在较强的接口依赖和时序依赖。因此，在过程定义中需要特别关注需求分析与系统设计，确保核心规则、模块边界、数据流转和接口调用关系在编码前得到充分说明。

此外，我们还将原型设计与可行性验证作为一个独立的过程加以保留。像 AI Agent 推理和语音交互功能所需要的技术比较难，不适合完全依赖一次性设计后直接实现。通过先构建大模型调用原型和语音接口验证原型，可以提前暴露技术风险，验证系统核心流程是否可行，并为后续详细设计和编码实现提供依据。

5.2 简化内容

项目对部分管理类过程进行了简化。首先是配置管理过程：项目仍然需要进行基本的代码版本管理、文档版本管理、需求变更记录和关键配置项管理，但由于团队规模小、项目周期有限，不需要建立复杂的配置管理委员会、正式变更审批流程和配置审计机制。因此，我们将配置管理简化为对代码仓库、文档版本、需求变更和重要设计修改的基本管理。

其次是质量保证过程：项目需要保证需求、设计、实现和测试之间的一致性，但我们并不设置独立的质量保证团队，质量保证活动主要通过文档评审、代码检查、接口联调、功能测试和教师阶段性反馈来完成。

再次是度量过程：我们只保留了必要的轻量化度量指标，例如功能完成度、核心需求覆盖情况、缺陷数量、接口联调通过情况、响应延迟和系统演示稳定性等，帮助团队判断项目过程是否有效，同时不会过度增加额外的管理负担。

对于系统的运行维护过程我们也进行了弱化处理。完整的软件生命周期通常包括长期运行、用户支持、版本升级、故障处理和系统退役等活动。但我们的主要目标是完成课程验收，并不面向真实商业用户长期运营。因此我们只保留了与课程验收相关的部署、演示、基础问题修复和后续改进建议，不将长期运维作为核心过程。

5.3 删除内容

由于本项目属于校内课程设计，项目团队既是开发方也是交付方，不存在正式的外部供应商关系、商业采购活动或合同约束，所以我们删除了采购、供应、合同管理和退役过程。同时，由于课程项目不会进入长期生产运行阶段，也不存在正式的软件退役需求，因此退役过程也不作为本项目软件过程的组成部分。